



Department of Computer Science and Engineering

JAMALPUR SCIENCE AND TECHNOLOGY UNIVERSITY

Project Report

SMARTMEDBOX

IoT-Based Smart Medicine Reminder and Monitoring System

Course Code

CSE 4132

Course Title

Internet of Things (IoT) Sessional

Submitted By:

Md. Naimur Rahman

ID: 22111111

Debbroto Kumar Karmokar

ID: 22111113

Nadia Nasrin Mukti

ID: 22211116

Mahbuba Akyer Joty

ID: 22211118

Himel Sarder

ID: 22111121

Md. Tanvir Ahmed

ID: 22111128

Session: 2021-2022

Department of Computer Science and
Engineering, JSTU

Submitted To:

Md. Khabir Uddin Ahamed

Lecturer

Department of Computer Science and
Engineering
Jamalpur Science and Technology
University, Jamalpur.

Submission Date: 16. 09. 26

Table of Contents

Section	Page
1. Introduction, Background and Problem Statement	3
1.1 Introduction	3
1.2 Background	3
1.3 Problem Statement	4
1.4 Objectives	5
2. Technology Used	5
3. Hardware Used	5
3.1 Component List	5
3.2 Key Component Images	6
3.3 Connection between Main Components	8
4. Methodology	9
5. Testing	10
5.1 Testing Process	10
5.2 Firmware and Connectivity Test	11
6. API Configuration	12
6.1 Core Features	13
6.2 System Architecture	13
6.3 Database Schema	13
6.4 API Endpoints	13
7. Hardware Assembly and Prototype	14
8. Mobile Application	15
9. Discussion	17
9.1 Advantages	17
9.2 Limitations	17
9.3 Future Improvements	18
10. Conclusion	18

1. Introduction, Background and Problem Statement

1.1 Introduction

Medication adherence is an important concern, especially for people who need to take medicines at specific times every day. Forgetting to take medicine, taking it at the wrong time, or being unable to confirm whether a dose has been taken can reduce the effectiveness of treatment. Traditional medicine boxes mainly provide a place to store medicines but generally do not provide real-time monitoring or confirmation of medication intake.

To address these limitations, this project presents SmartMedBox, an IoT-based smart medicine box designed to assist users in monitoring their medication activities. The system combines an ESP32 microcontroller, Reed switch, IR sensor, DS3231 RTC, LEDs, and a physical Medicine Taken push button. The Reed switch detects whether the medicine box is opened, while the IR sensor helps detect the presence or removal of medicine inside the box. The DS3231 RTC provides accurate time information for recording medication-related activities.

The ESP32 collects sensor information and sends it to a cloud-based backend through Wi-Fi. The backend is developed using FastAPI and stores the collected sensor records in a database. A Flutter-based mobile application, DoseMate, communicates with the backend API and allows users to monitor the condition of the medicine box and medication-related activities.

An important feature of the system is the physical Medicine Taken push button, which works as a manual emergency or fallback mechanism. In some situations, the Reed switch or IR sensor may fail to detect the medication-taking activity correctly because of sensor limitations, positioning issues, or other technical problems. In such cases, the user can manually press the push button to confirm that the medicine has been taken. The ESP32 then sends a `taken=true` event to the backend, regardless of the current Reed or IR sensor state. The DoseMate application can identify this event, display a “Medicine Taken” status, record it in the medication history, and provide a notification to the user.

Overall, SmartMedBox provides a connected solution for medication monitoring by integrating embedded hardware, cloud-based data storage, and a mobile application. The combination of automatic sensor-based detection and a manual emergency confirmation mechanism makes the system more flexible and reliable for monitoring medication-taking activities.

1.2 Background

Medication management has become an important issue for people who need to take medicines regularly, particularly elderly patients, people with chronic diseases, and individuals who take multiple medicines each day. Missing a dose, taking medicine at the wrong time, or accidentally taking a dose more than once can

negatively affect treatment. In many cases, patients or caregivers also have difficulty confirming whether a medicine has actually been taken, especially when the patient is alone.

Traditional medicine boxes can help organize and store medicines, but they generally do not provide any mechanism for monitoring medication-related activities. Some reminder applications can notify users about scheduled doses, but a reminder alone cannot confirm whether the medicine box was opened or whether the medicine was actually taken. Therefore, there is a need for a system that can combine medication reminders with real-time monitoring and activity confirmation.

The development of the Internet of Things (IoT) has created opportunities to improve healthcare-related systems by connecting physical devices with software applications and cloud-based services. Sensors connected to a microcontroller can collect information from the physical environment and transmit it to a remote server. This information can then be stored, processed, and presented to users through a mobile application. Such an approach can make medication monitoring more interactive and provide useful information to both users and caregivers.

In SmartMedBox, different sensors are used to monitor the medicine box. A Reed switch is used to detect whether the box is opened or closed, while an IR sensor provides additional information about the presence or movement of medicine inside the box. A DS3231 Real-Time Clock (RTC) is used to maintain accurate time information for recording activities. The ESP32 acts as the main controller, collects the sensor data, and communicates with the backend server through Wi-Fi.

However, sensor-based detection may not always be reliable. Sensor positioning, environmental conditions, hardware faults, or unexpected user actions may cause the system to miss an actual medication-taking event. To address this limitation, SmartMedBox includes a physical Medicine Taken push button as an emergency or fallback mechanism. If the automatic sensors fail to detect the activity correctly, the user can manually press the button to confirm that the medicine has been taken. This confirmation is sent independently of the Reed and IR sensor states, allowing the system to record the event even when automatic detection is unavailable.

The collected information is transmitted to a FastAPI-based backend and stored in a database. The DoseMate Flutter application retrieves the information through the API and presents the current box status and medication-related events to the user. This combination of sensor-based monitoring, manual emergency confirmation, cloud-based data management, and mobile access provides the foundation for a more practical and connected medication monitoring system.

1.3 Problem Statement

The following problems motivate this project:

- Users may forget to take medicine at the scheduled time.
- A conventional medicine box cannot remind the user.
- Family members or caregivers may not know whether medicine has been taken.

- It is difficult to automatically record medication events.
- Weight-based medicine detection alone may produce unreliable results.
- A low-cost, easily reproducible solution is required for prototype development.

Therefore, the project aims to design a system that provides local and remote reminders and reliably detects medicine-taking activity by combining readings from multiple sensors rather than relying on a single sensor.

1.4 Objectives

- Design a low-cost smart medicine reminder system.
- Use an RTC for accurate, offline-capable medication scheduling.
- Provide local reminders using a buzzer and an LED.
- Detect whether the medicine box has been opened, using a reed switch.
- Detect activity inside the box using an IR sensor.
- Detect medicine removal using a load cell and HX711 amplifier.
- Fuse multiple sensor readings for reliable medicine-taking detection.
- Transmit medication status to the cloud (Firebase) over Wi-Fi.
- Display medication status and history through an Android mobile application.

2. Technology Used

The project integrates embedded firmware, wireless communication and cloud/mobile technologies as summarized below.

Technology	Purpose
Arduino IDE	ESP32 firmware development environment
C / C++	Embedded firmware programming language
ESP32 (Dev Module)	Main controller with built-in Wi-Fi
DS3231 RTC	Accurate, battery-backed date and time keeping
HX711 (Proposed)	24-bit ADC / load-cell signal amplification
Wi-Fi	Wireless connectivity for cloud synchronization
Android / Flutter	Mobile application for remote monitoring
Text-to-Speech (TTS)	Voice reminders through the mobile app
FastAPI (Python) + PostgreSQL	Backend API for storing and serving sensor data (test/development)
GitHub	Source-code version control and repository management

3. Hardware Used

3.1 Component List

No.	Component	Qty	Function
1	ESP32 ESP-32S 30-Pin NodeMCU Development Board	1	Main controller / Wi-Fi
2	DS3231 RTC Module (AT24C32 EEPROM, I2C)	1	Real-time clock
3	5 kg Weighing Load Cell (Proposed)	1	Weight sensing
4	HX711 Load-Cell Amplifier Module (Proposed)	1	Load-cell signal amplification / ADC
5	Reed Switch (Normally Open) + Magnet	1	Box open/close detection
6	Infrared (IR) Obstacle/Activity Sensor Module	1	Internal activity detection
7	3.3V-Compatible Active Buzzer	1	Local audio reminder
8	LED (5 mm)	1	Visual reminder indicator
9	220 Ω Resistor	1	LED current-limiting resistor
10	Push Button Switch	1	Manual acknowledgement
11	Breadboard (Half Size, 400 tie points)	1	Prototyping
12	Jumper Wires (Male-to-Female / Male-to-Male)	As required	Circuit connections
13	USB Cable (Micro-B)	1	Programming / power
14	5V USB Adapter / Power Bank	1	Power supply
15	Prototype Medicine Box (enclosure)	1	Physical housing

Note: An OLED display, MP3/WAV player and speaker were considered but intentionally excluded from the final simplified design; the mobile application handles detailed display and optional voice notification instead, using smartphone Text-to-Speech.

3.2 Key Component Images



Figure 1: ESP32 ESP-32S 30-Pin NodeMCU Development Board (Wireless Wi-Fi)



Figure 2: Precision Real-Time Clock Module — DS3231 with AT24C32 EEPROM (I2C)

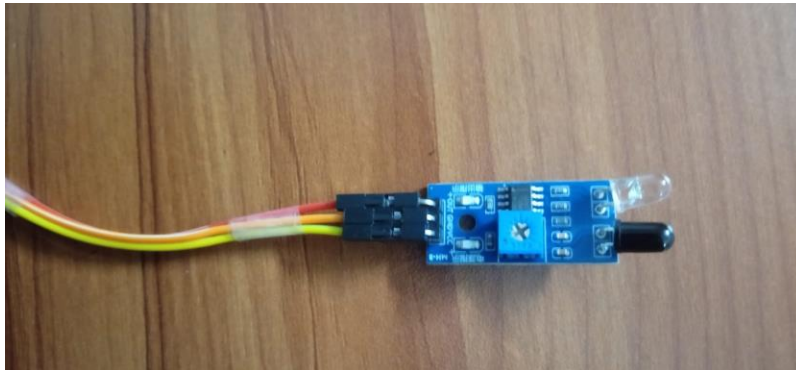


Figure 3: Infrared (IR) Obstacle Avoidance / Activity Sensor Module

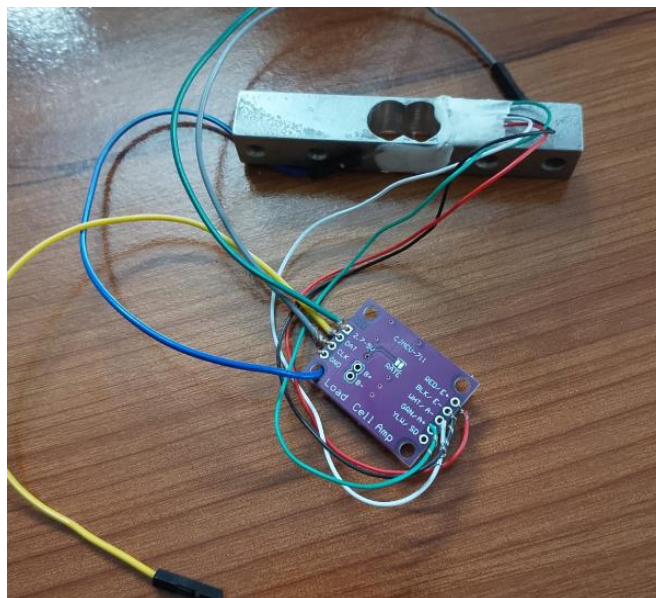


Figure 4: 5 kg Load Cell wired to the HX711 Load-Cell Amplifier Module



Figure 5: Reed Switch

3.3 Connection between main components

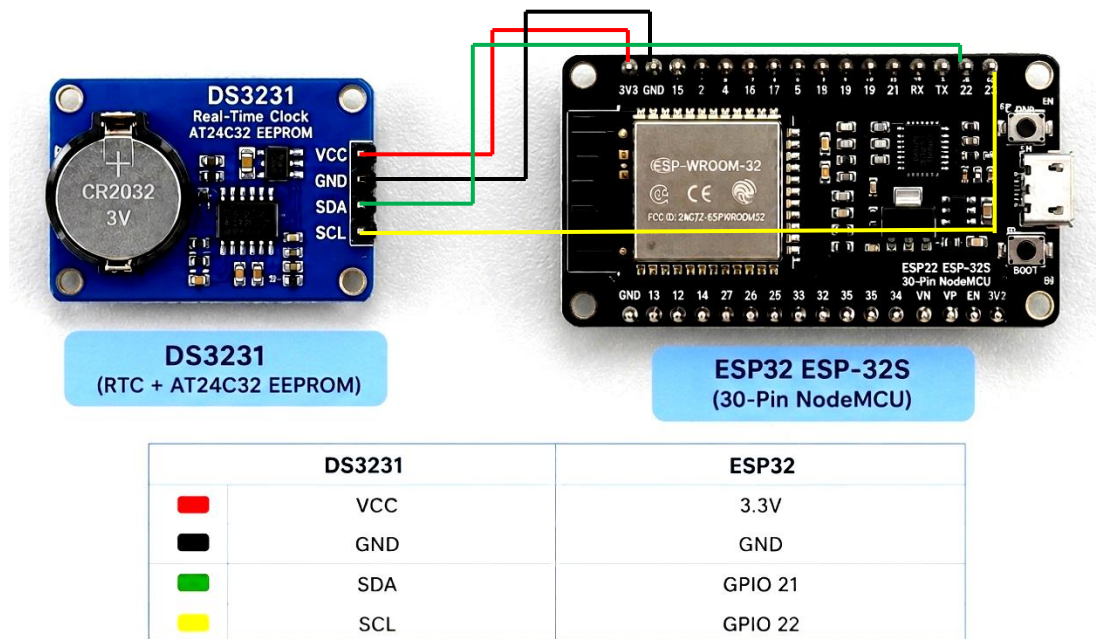
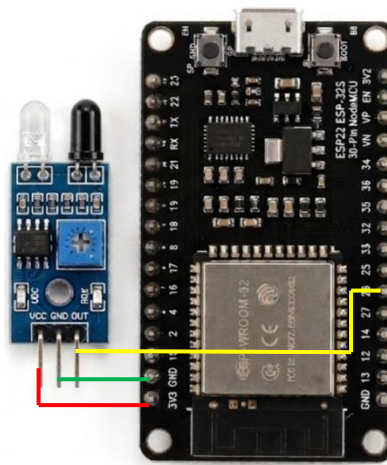
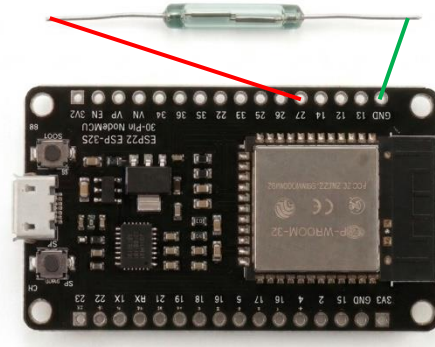


Figure 6: Connection between DS3231 and ESP32



IR Sensor	ESP32
VCC	3.3V*
GND	GND
OUT	GPIO 26

Figure 7: Connection between IR sensor and ESP32



GPIO27 — Reed Switch — GND

Figure 8: Connection between Reed Switch and ESP32

Component	ESP32 Pin
DS3231 SDA	GPIO 21
DS3231 SCL	GPIO 22
HX711 DT / DOUT	GPIO 32
HX711 SCK	GPIO 33
Reed Switch	GPIO 27
IR Sensor OUT	GPIO 26
Active Buzzer	GPIO 25
LED	GPIO 2
Push Button	GPIO 14

4. Methodology

The proposed SmartMedBox system works by integrating an IoT-enabled medicine box with a cloud-based backend and the DoseMate mobile application. At first, the user sets the required medication schedule through the mobile application. During the scheduled medication period, the sensors connected to the ESP32 continuously monitor the condition of the medicine box. The Reed switch detects whether the box is opened or closed, while the IR sensor detects the presence or movement of medicine inside the box. The DS3231 RTC provides accurate date and time information for recording each activity. An HX711-based load cell can also be used to monitor changes in the medicine-box weight. In addition to the automatic sensors, a physical Medicine

Taken push button is included as a manual or emergency confirmation mechanism. If the Reed or IR sensor fails to detect the medication activity correctly, the user can press this button to directly confirm that the medicine has been taken. The ESP32 processes the collected sensor information and sends it to the FastAPI backend through Wi-Fi using HTTP and JSON communication. The backend validates and stores the received information in the database, including the box status, IR detection status, medicine-taken event, weight, and RTC timestamp. The DoseMate Flutter application retrieves the latest information from the backend and presents the current condition of the medicine box to the user. When a new medicine-taken event is detected, either through the sensor-based mechanism or the physical button, the application records the event in the medication history and can provide notification and voice feedback. The system therefore creates a complete flow from sensor-based monitoring and manual confirmation to cloud storage and mobile visualization, allowing medication activities to be monitored in an integrated and convenient manner.

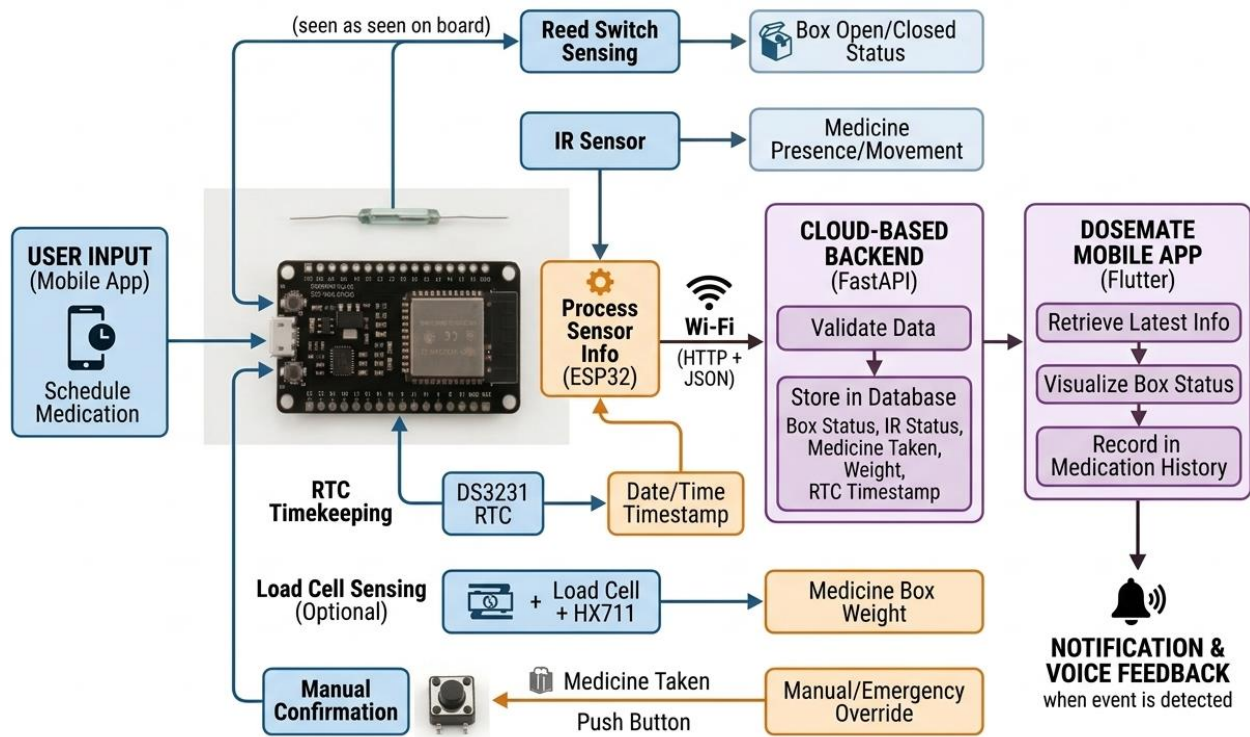


Figure 9: Methodology Flow Diagram

5. Testing

5.1 Testing Process

To manage complexity, the firmware was developed and validated incrementally rather than as one large program, following this sequence:

- ESP32 board setup and Blink test
- Serial communication test

- DS3231 RTC test
- HX711 test and load-cell calibration
- Reed switch test
- IR sensor test
- LED and push-button tests
- Combined multi-sensor test
- Wi-Fi connectivity test
- Firebase / backend API integration test
- Mobile application integration
- Final system integration

Test	Expected Result	Actual Result
ESP32 board upload	Successful	Successful
DS3231 RTC detection	RTC detected	RTC detected
RTC time reading	Correct time displayed	Correct time displayed
HX711 detection	HX711 detected	HX711 detected
Load cell reading	Weight changes detected	Sensor not working
Reed switch	Open / close detected	Open / close detected
IR sensor	Activity detected	Activity detected
LED	Visual reminder generated	Visual reminder generated
Push button	Manual input detected	Manual input detected
Wi-Fi connectivity	ESP32 connected	ESP32 connected (IP obtained)
Backend / cloud transmission	Data transmitted successfully	HTTP 200 — sensor data saved

5.1 Firmware and Connectivity Test

After Wi-Fi connectivity was established, the ESP32 successfully transmitted a JSON-formatted sensor payload (device ID, box-open status, IR-detection status, weight and RTC timestamp) to the backend API, which returned an HTTP 200 response confirming that the sensor record was saved, as captured in Figure 10.

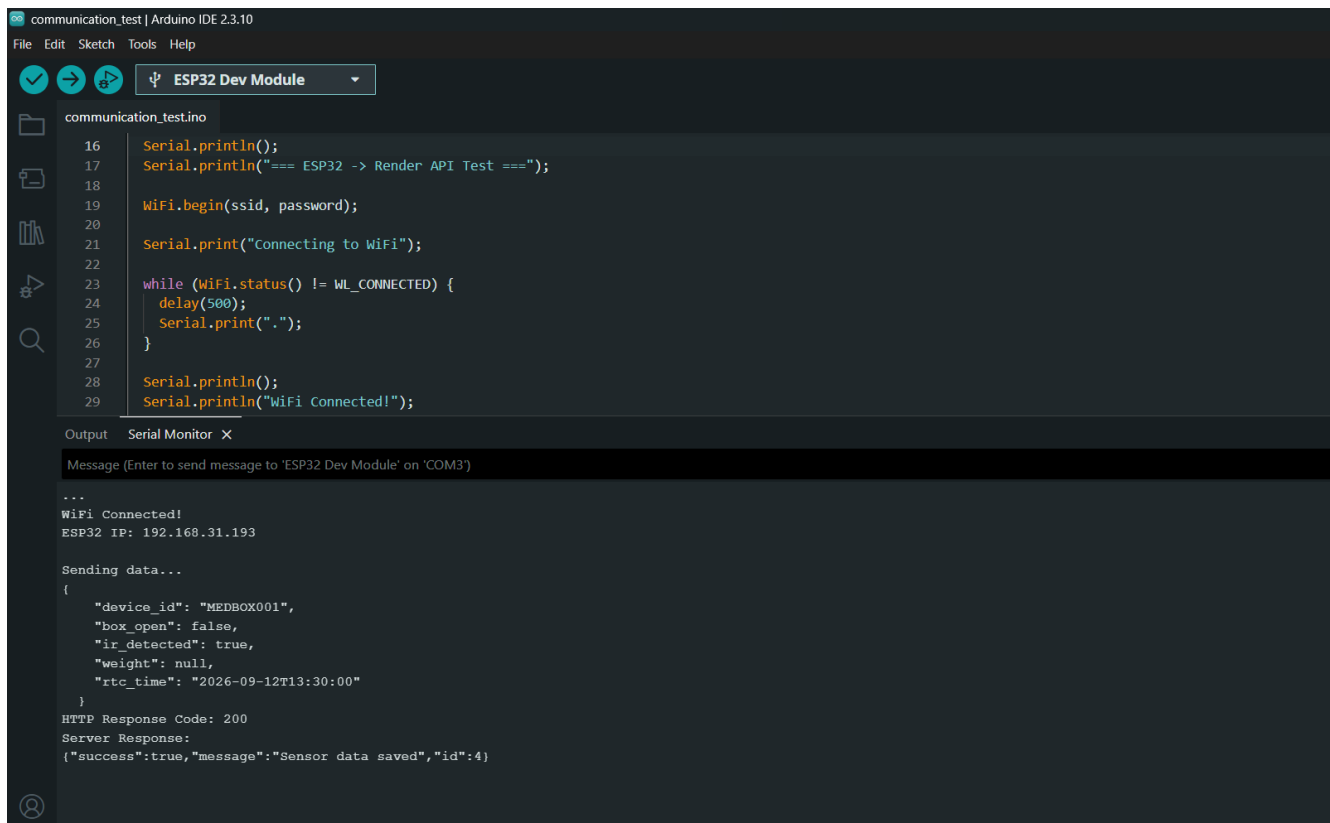


Figure 10: Arduino IDE Serial Monitor Output Showing Successful Wi-Fi Connection and Sensor-Data Transmission

6. API configuration

SmartMedBox Backend is a robust, asynchronous REST API built with FastAPI. It serves as the central hub for the SmartMedBox ecosystem - a smart medication adherence and tracking system. The backend receives, validates, and stores telemetry data from IoT edge devices (such as an ESP32 or Raspberry Pi) attached to physical medication boxes, and serves that data to end-user applications for monitoring.

SmartMedBox API 1.0.0 OAS 3.1

/openapi.json

default		^
GET	/ Root	▼
GET	/health Health	▼
POST	/api/sensor Receive Sensor Data	▼
GET	/api/sensor/latest Get Latest Sensor Data	▼

Figure 11: SmartMedBox FastAPI docs

6.1 Core Features

- **High-Performance API:** Powered by FastAPI and Uvicorn for asynchronous, fast HTTP request handling.

- **Data Integrity:** Uses Pydantic schemas to strictly validate incoming JSON payloads from hardware devices.
- **Relational Storage:** Employs SQLAlchemy ORM to manage database records, making it compatible with SQLite (for development) and PostgreSQL/MySQL (for production).
- **Time-Series Tracking:** Accurately tracks events based on device-side Real-Time Clock (RTC) timestamps to ensure data consistency even during network outages.

6.2 System Architecture

The diagram below illustrates the high-level architecture, showing how the hardware components interact with the backend API and the user-facing application:

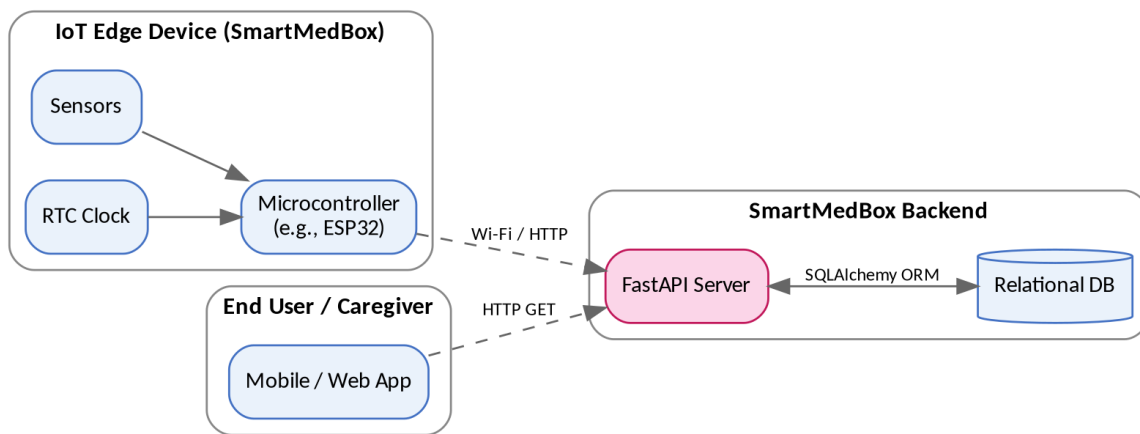


Figure 12: System to API Architecture

6.3 Database Schema

The application uses the following entity-relationship schema for storing telemetry (table `SENSOR_DATA`):

Field	Type	Description
id	Integer (PK)	Auto-incrementing primary key
device_id	String	Unique identifier for the physical box
box_open	Boolean	State of the lid
ir_detected	Boolean	Hand detection state
weight	Float	Current weight in grams/oz
rtc_time	DateTime	Time recorded by device RTC
created_at	DateTime	Server-side insertion timestamp

6.4 API Endpoints

Method	Endpoint	Description
GET	/	Root endpoint, returns API status message.
GET	/health	Health check endpoint for uptime monitoring.
POST	/api/sensor	Submit new sensor data from the IoT device.
GET	/api/sensor/latest	Retrieve the most recent sensor data entry for dashboard display.

7. Hardware Assembly and Prototype

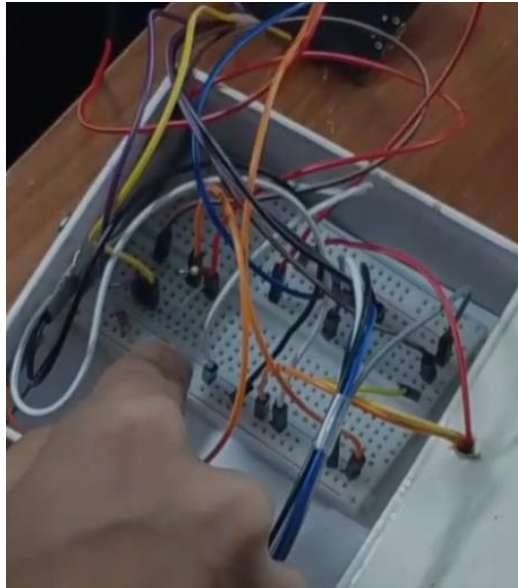


Figure 13: Breadboard Wiring of Sensors Inside the Prototype Enclosure



Figure 14: Assembling and Securing the Wiring Inside the Medicine Box Enclosure

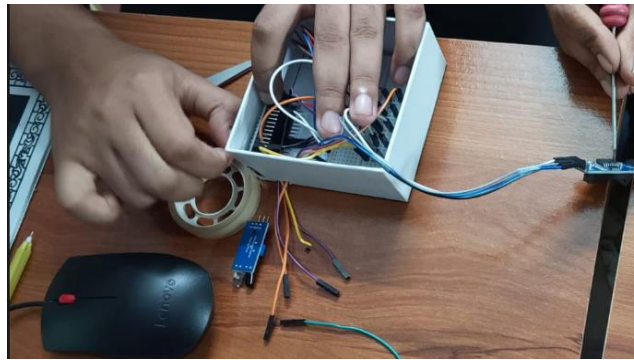


Figure 15: Completed SmartMedBox Prototype with IR Sensor Mounted on the Lid and Status LEDs



Figure 16: SmartMedBox is in power mood.

Overall, the developed SmartMedBox prototype successfully demonstrated the integration of multiple sensors with an ESP32 controller. The DS3231 provided the required time information, the reed switch detected box opening, and the IR sensor detected internal activity. The load cell and HX711 identified changes in medicine weight, while the buzzer and LED provided local reminders. After establishing Wi-Fi connectivity, the system transmitted monitoring information to the backend/cloud platform, confirming end-to-end functionality from sensing to data storage.

8. Mobile Application

Key features

- Medicine scheduling — name, dosage, type (tablet/capsule/syrup/injection/other), instructions (before/after meal, empty stomach, as needed), one or more times a day, daily or specific-weekday repeats, start/end dates.
- Four-stage reminder cycle per dose (early reminder → due → late warning → missed).



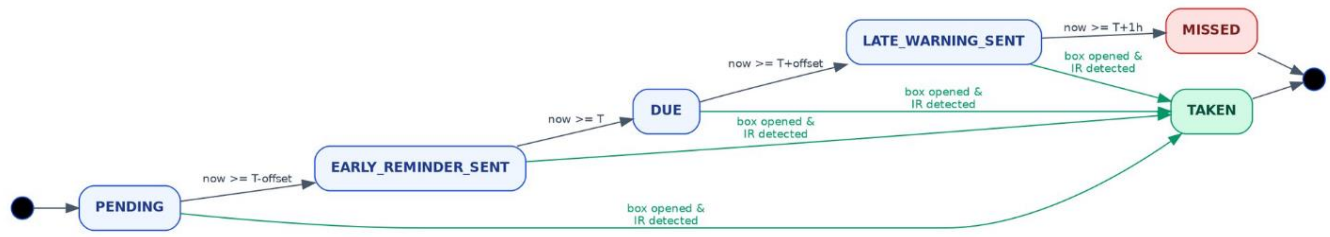


Figure 17: 4 stages of reminder

- Full-screen ringing alarm for every stage — loops a native alarm tone and vibrates continuously until dismissed or the dose is confirmed taken, and shows over the lock screen (fullScreenIntent + custom window flags), like a real alarm clock rather than a quiet notification.
- Automatic dose confirmation from the SmartMedBox IoT device — never relies on the user self-reporting.
- Offline-first local history — every medicine, schedule, and dose log lives in a local SQLite (Drift) database; the backend is only consulted for the box's sensor state.
- Device-mismatch protection — if the backend's latest reading belongs to a different Device ID than the one configured in Settings, the app surfaces a clear warning banner instead of silently attributing someone else's box event to you.
- Configurable timing — the early-reminder/late-warning offset (5 or 10 minutes) is user-adjustable in Settings; changing it live-reschedules every pending notification.
- Bangla-first UI with light/dark theme (follows system by default).

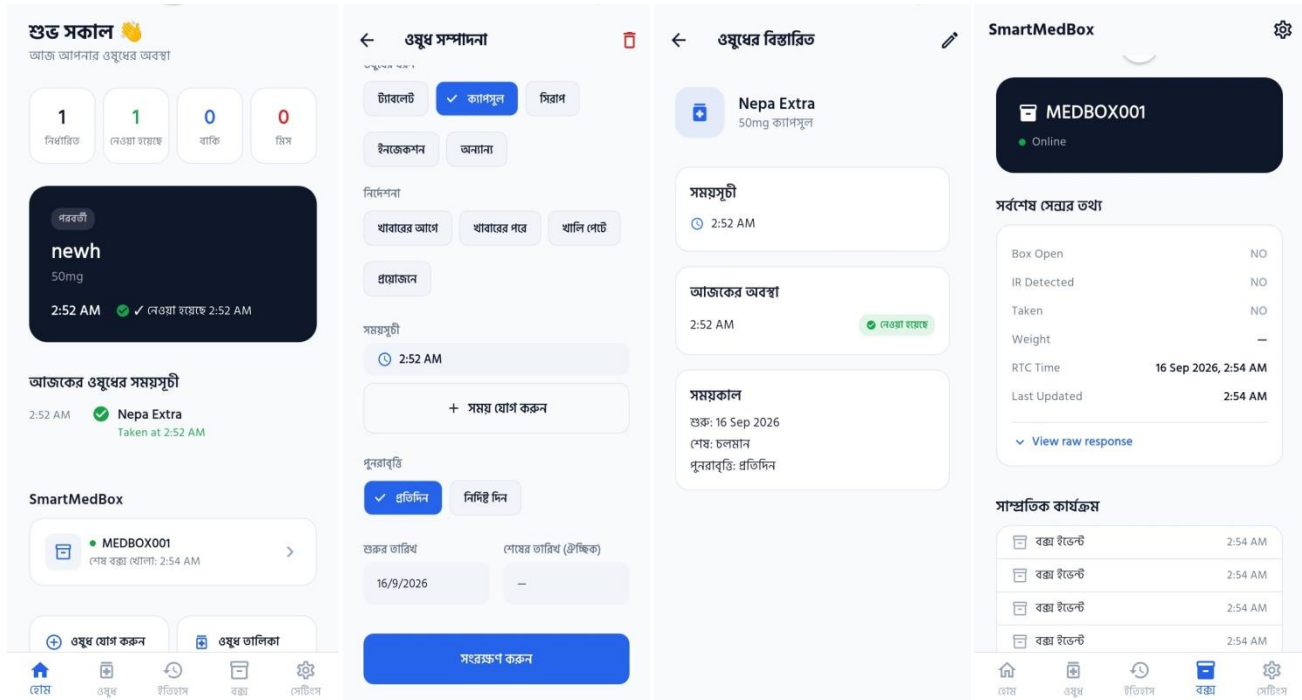


Figure 18: App Interface

9. Discussion

The proposed SmartMedBox demonstrates how IoT and embedded sensing technologies can be combined to develop a low-cost medication-assistance system.

One important design decision is the use of multiple sensors rather than a single one. A load cell alone may not always provide reliable medication detection because weight measurements can be affected by vibration, sensor mounting, container movement and small medicine-weight differences. The system therefore combines weight change with box-opening and IR-activity information, so that "medicine taken" is confirmed only when all three signals agree.

The DS3231 provides an independent, battery-backed real-time clock, allowing medication schedules to be maintained without depending entirely on an internet connection for timekeeping. The ESP32 is a suitable platform because it combines microcontroller functionality with built-in Wi-Fi, removing the need for a separate wireless module.

Removing the OLED display, MP3 player and speaker simplified the hardware design and reduced cost. The mobile application instead provides detailed information and optional voice notifications using the smartphone's own Text-to-Speech engine, while the local buzzer and LED remain sufficient for an immediate physical reminder.

The prototype does, however, have some practical limitations. A 5 kg load cell is rated for loads up to its full-scale capacity, and the minimum reliably detectable weight change depends on the complete mechanical setup and calibration; very small tablet-weight changes can therefore be difficult to detect consistently without careful sensor placement, calibration and filtering. The current prototype is also designed for a single medicine compartment; a future version could support multiple compartments and multiple medication schedules.

9.1 Advantages

- Low-cost hardware architecture
- Automated, time-scheduled medication reminders
- Real-time local and remote monitoring
- Multiple-sensor-based (fused) detection for higher reliability
- Wireless connectivity with cloud-based data storage
- Mobile application monitoring with medication history
- Optional voice reminders through the smartphone
- Modular and expandable architecture

9.2 Limitations

- Designed for a single medicine compartment in this prototype stage
- Weight sensor didn't work for technical issues.

- Load-cell accuracy depends on calibration and mechanical installation
- Wi-Fi connectivity is required for cloud synchronization
- IR-sensor detection accuracy depends on sensor placement
- The reed switch only indicates box opening/closing, not who opened it
- The prototype does not identify the exact person who took the medicine
- This is a prototype and should not be considered a medical-grade device

9.3 Future Improvements

- Multi-compartment support (e.g., Morning / Noon / Night compartments)
- Mobile-app-based schedule configuration by the user
- Caregiver monitoring with missed-medication alerts
- Lower-capacity / higher-resolution load cell for smaller weight changes
- Machine-learning classification of sensor patterns (Taken / Missed / Opened-but-not-taken / False interaction)
- Additional notification channels such as push notifications or SMS

10. Conclusion

The SmartMedBox project presents a low-cost IoT-based approach to medication reminder and monitoring. The system uses an ESP32 as the main controller and integrates a DS3231 RTC, a load cell with HX711 amplifier, a reed switch, an IR sensor, a buzzer, an LED and a push button.

The DS3231 provides time-based scheduling, while the buzzer and LED provide local reminders. The reed switch detects box opening, the IR sensor detects internal activity, and the load cell with HX711 monitors weight changes; by fusing these sensor readings the system produces a more reliable estimate of whether medication has actually been taken. The ESP32's built-in Wi-Fi enables communication with Firebase and a backend API, and the companion mobile application offers a convenient interface for viewing medication schedules, status and history, with optional smartphone Text-to-Speech voice reminders.

Overall, the project demonstrates the practical application of IoT, embedded systems, sensor fusion, cloud computing and mobile application development in a healthcare-support context. Although the current prototype has some limitations, its modular architecture allows future expansion into a multi-compartment and more intelligent medication-management system.